# UC-47.03 — Approve Profile Update Request

Administrator
:ProfileRevisionAdminView
:ProfileRevisionAdminRoute
:ProfileRevisionAdminController
:ProfileRevisionService
:ProfileRevisionRepository
:PrismaService
Database
1. Approve request; optional review notes
2. PATCH /admin/profile-revisions/:id/approve (body)
3. approve(id, adminId, reviewNotes) -> review(...)
ref
3.1 Verify jwt
break
[invalid token / session / role permission]
3.1.1. 401 / 403 error response
3.1.2. Display authentication / permission error
ref
3.2. PermissionGuard: roleHasPermission(roleId, method, path)
break
[request DTO / id validation fails]
3.3.1. 400 validation error
3.3.2. Display validation message
4. approve(id, adminId, reviewNotes) -> review(...)
5. findById(id)
6. profileRevision.findUnique(id; revisionSelect)
7. SELECT revision with user/admin identity and snapshot data
8. Revision or null
9. Revision or null
10. Revision or null
break
[revision missing]
11.1. 404 ProfileRevisionNotFoundException
11.2. Propagate HTTP error
11.3. Return error response
11.4. Display error message
break
[revision.status != PENDING]
12.1. 409 ProfileRevisionAlreadyReviewedException
12.2. Propagate HTTP error
12.3. Return error response
12.4. Display error message
13. Trim reviewNotes; parse proposedData with schema selected by FREELANCER / CLIENT / EXPERT
ref
ProfileRevisionRepository.review: all claim, profile, notification and final-read operations below are one interactive Prisma transaction
14. review: claim PENDING revision as APPROVED
15. transaction.profileRevision.updateMany(id; status PENDING -> APPROVED; adminId, reviewNotes, reviewedAt)
16. Conditional UPDATE pending revision; prevents concurrent double review
17. { count }
18. { count }
19. { count }
break
[claim count == 0: repository returns null]
20.1. 409 ProfileRevisionAlreadyReviewedException
20.2. Propagate HTTP error
20.3. Return error response
20.4. Display error message
21. review: load target profile and user
22. transaction.profileRevision.findUniqueOrThrow(id; select profileId,userId)
23. SELECT revision profileId and userId
24. Revision target
25. Revision target
26. Revision target
27. review: apply proposedData by profileType
28. transaction.profile.update; freelancerProfile / clientProfile / expert.upsert
29. UPDATE base Profile; UPSERT matching role profile with approved fields
30. Updated role profile
31. Updated role profile
32. Updated role profile
33. review: recompute completion strength
34. transaction.profile.findUniqueOrThrow(include role profiles, skill/active portfolio counts); profile.update
35. SELECT updated profile; calculate role-specific strength; UPDATE profileCompletionPercent
36. Updated strength
37. Updated strength
38. Updated strength
39. review: notify owner and return result
40. transaction.notification.create; profileRevision.findUnique(revisionSelect)
41. INSERT SYSTEM_ANNOUNCEMENT kind PROFILE_APPROVED; SELECT reviewed revision; COMMIT
42. Reviewed revision
43. Reviewed revision
44. Reviewed revision
45. ProfileRevisionDetailDto
46. Return controller result
47. 200 OK: ProfileRevisionDetailDto
48. Display reviewed request; refresh administrative list